

Python

> : 2026-08-19 15:59:01

- 1 Python
 - 2
 - 3
 - 4
 - 5 List
 - 6
 - 7
 - 8 I/O
 - 9
 - 10 OOP
 - 11
 - 12
-

Python 1

- Linux “ ” “ ”
-

- - 4200
-

1 Python

Python
C Java Python Python
C C++ Java Python
Python
Windows macOS Linux Python pip IDE—Visual Studio
Code Python
“ ”

1.1 Python

Python Guido van Rossum · 1989
1991
Monty Python Python
Python “The Zen of Python” Python `import this`

> Beautiful is better than ugly. > Simple is better than complex.
> Readability counts.

Python {}
Python “ ”

World” “ Python 10 “Hello

1.2 Python

1. C/C++ “ ” 2. Python
Windows macOS Linux type hints 3. Python
4. Batteries Included
Python “ ” requests+BeautifulSoup
pandas scikit-learn PyTorch Web Django FastAPI

- C/C++ Python 10
- Java Java 1/5~1/3
- JavaScript

Python

- Python CPU
Python
- pandas Excel matplotlib Jupyter Notebook
- ChatGPT Python TensorFlow PyTorch Python
- **Web** Django Flask/FastAPI API
-
- Pygame PyQt

/ Python Python
“ ” —

1.3 Python

Python 3.11 3.12

3.12

Python 2.7

Windows

1. <https://www.python.org/downloads/>
2. Python 3.12.x Windows installer
3. **“Add python.exe to PATH”**
4. “Install Now”
5. Python Win+R `cmd`

```
python --version
pip --version
```

Python 3.12.5 pip 24.x.x

macOS

Homebrew

Homebrew

```
brew install python
```

pkg python3 --version macOS python 2.x python3 pip3

Linux Ubuntu

```
sudo apt update
sudo apt install python3 python3-pip -y
```

Linux Python3 pyenv Python

Python ~/my_python_project

1.4 pip — Python

pip Python

Linux apt Node.js npm


```
#
```

```
pip --help
```

```
pip install requests
```

```
pip install requests==2.31.0
```

pip

```
python -m pip install --upgrade pip
```

```
pip uninstall requests
```

```
pip list
```

requirements.txt

```
pip freeze > requirements.txt
```

```
pip install -r requirements.txt
```

PyPI

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple somepackage
```

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

venv

```
python -m venv venv
```

Windows

```
venv\Scripts\activate
```

macOS/Linux

```
source venv/bin/activate
```

```
(venv)
```

1.5 IDE — Visual Studio Code

Python IDLE

VS Code

“ ”

1. <https://code.visualstudio.com/> 2. VS Code `Ctrl+Shift+X`

- **Python**

- **Pylance** Python

-

Black Formatter
Language Pack

- **isort**

import - **Chinese (Simplified)**

3. Python `Ctrl+Shift+P` “Python: Select Interpreter”

Python 3.12

4.

`Ctrl+,`

-

`"editor.formatOnSave": true`

- `"python.linting.enabled":`

`true` - Black

Python

VS Code

`.py`

F5

1.6 Python

6.1 REPL - Read-Eval-Print-Loop

`python`

`python3`

`>>>`

Python “ ”


```
#  
print("Hello, Python!      ")
```

```
name = input("      ")  
print(f"    {name}      ")
```

```
print("1 + 2 = ", 1 + 2)  
print(5 * (3 + 7))
```

REPL

- Read
- Eval
- Print
- Loop

`exit()` `Ctrl+D` Linux/macOS `Ctrl+Z` Windows

6.2

`hello.py`

```
#!/usr/bin/env python3
```

```
-- coding: utf-8 --
```

```
"""
1    Python

    2026
"""

def greet(name):
    """    """
    print(f"        {name} ")
    print("    Python    ")
    print("        Python ")
    print("=" * 30)

if __name__ == "__main__":
    print("        ...")
    user_name = input("        ")
    greet(user_name)

#
a = int(input("        "))
b = int(input("        "))
print(f"{a} + {b} = {a + b}")
print("        ")
```

1.

```
python hello.py
```

2. VS Code “Run Python File in Terminal”

- `.py`
- `# -- coding: utf-8 --` Python 3 utf-8
- `if __name__ == "__main__":` Python “ ”
import
- **4 Tab** IndentationError

1.7

- `'python'` Python PATH Add to PATH
 - UnicodeDecodeError UTF-8
 - pip
 - VS Code Python Ctrl+Shift+P
 - 1.
 - 10 2. `print()` “ ” 3.
 - 4.
-

- Python
- Python pip
- pip
- VS Code Python
- REPL Python
 - “ ” Python Python

1. `hello.py` 2. `pip rich` `pip install`
`rich` `from rich.console import Console` 3. VS Code
F5

2 Python

4200

2

2

2

Python 2

“ ”
Python

Linux “ ”

BMI

2.1

5 2 1 “ ”

Python

-
-
- Python

```
# 2
apple_price = 8.5      #
apple_count = 5        #
total_price = apple_price * apple_count

print("      :", total_price, " ")
```

: 42.5

apple_price apple_count total_price

2.2

“ ” Python

2.2.1

1. a-z A-Z 0-9 _ 2. 3. Age age
4. **Python** if else for while class def True
False None

- user_name
- total_price_2025
- count
- is_valid
- PI

- `2total` →
- `total-price` →
- `class` →
- `user name` →

2.2.2

`a=10` `b=20`

-
- `snake_case` + `user_age` `max_score`
- `MAX_USERS = 100`
- `i` `j` `x` `y`
- Python3

`Ctrl +` VS Code/PyCharm

2.3 Python

Python `=` `=` “ ”

3.1


```
#  
a = 10  
b = 20  
c = a + b
```

Pythonic

```
x, y, z = 1, 2, 3  
print(x, y, z) # : 1 2 3
```

```
x, y = y, x
print("    x={}, y={}".format(x, y))
```

```
1 2 3
    x=2, y=1
```

3.2 Python

Python

```
#
var = 100
print("var    ", var, "    ", type(var))

var = 3.14159
print("var    ", var, "    ", type(var))

var = "        "
print("var    ", var, "    ", type(var))

var = True
print("var    ", var, "    ", type(var))
```

```
var    100
var    3.14159
var
var    True
```

Python

`type()` `isinstance()`

2.4 int float

Python3 `int`

Python2 `int` `long`

4.1 int

```
# Python
big_number = 123456789012345678901234567890
print(big_number * 2)
```

```
print(10 + 5)    # 15
print(10 - 5)    # 5
print(10 * 5)    # 50
print(10 // 3)   # 3
print(10 % 3)    # 1
print(2 ** 8)    # 256
```

4.2 float

```
#  
print(0.1 + 0.2)          #    0.30000000000000004
```

1 round()

```
print(round(0.1 + 0.2, 1)) #    0.3
```

2 decimal

```
from decimal import Decimal, getcontext
getcontext().prec = 28
a = Decimal('0.1')
b = Decimal('0.2')
print(a + b)           # 0.3
```

float

2.5 bool

True False

```
is_adult = True
has_license = False

print(type(is_adult)) #
```

```
print(True == 1)      # True
print(False == 0)     # True
```

False

- False
- 0 0.0
- ""

- `[]` `{}` `()`
- `None`
- `True`

2.6 `int()` `float()` `bool()` `str()`

```
#
age_str = "25"
age = int(age_str)
print(age, type(age))

height = 175.8
height_int = int(height)      # 175
print(height_int)

score = 0
print(bool(score))            # False
```

```
user_input = input("      ") # input
user_age = int(user_input)
print("      ", user_age + 1, " ")
```

- `int("abc")` → `ValueError`
- `try...except` `user_input.isdigit()`

2.7

7.1

+ - / // % *

```
total = 100
total += 20    #      total = total + 20
total *= 2
print(total)  # 240
```

7.2

== != > >= < <=

```
x = 10
y = 20
print(x > y)      # False
print(x <= 10)    # True
print(1 < x < 20)  # Python
```

7.3

and or not

```
age = 25
has_money = True

if age >= 18 and has_money:
    print("    ")
```

```
print(True or print("    ")) # print
```

> > > >

()

2.8

1 BMI

```
# BMI -  
height = float(input(" "))  
weight = float(input(" "))  
  
bmi = weight / (height ** 2)  
print(" BMI {:.2f}".format(bmi))  
  
if bmi < 18.5:  
    print(" ")  
elif bmi < 24:  
    print(" ")  
elif bmi < 28:  
    print(" ")  
else:  
    print(" ")
```

2

```
principal = 10000.0
rate = 0.035          #    3.5%
years = 5

final_amount = principal * (1 + rate) ** years
interest = final_amount - principal

print(f"Principal: {principal}, Years: {years}, Final Amount: {final_amount:.2f}")
print(f"Interest: {interest:.2f}")
```

2.9

1. **NameError: name 'total' is not defined** →
2. **TypeError: can only concatenate str** →
3. **ZeroDivisionError** →

0 4. → ==

```
print(" " " ")
```

-
- `int` `float` `bool`
-

-
-
-

“ ”

1. `pi = 3.14159` 2. `C = (F - 32) * 5/9` 3. `%` 4. `1000 9 2000 8`
 5. `3`

Python

3800+

Python 3.12

3

Python

Python

3

“ ”

— “ ” “ ”

- `if-elif-else`
- `while` `for...in`

- `break` `continue` `pass`

“ ”

“ ” “ ”

3.1 if-elif-else

3.1.1

Python “ ” **True** **False**
bool

- `>` `>=` `<` `<=` `==` `!=`
- `and` `or` `not`

Python `False` `0` `None` `''` `[]`

3.1.2

`if`

```
if :
    # True
    print(" ")
```

1

```
# odd_even.py
number = int(input("Enter a number: "))

if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")

print("Program ended")
```

```
17
17
```

```
number % 2 == 0
```

3.1.3 if-elif-else

```
elif else if
```

1

4

```
# score_level.py
score = float(input("Enter score (0-100) "))

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("...")
```

```
if score >= 60 and score < 70:
    print("C")
```

- `if` `elif` `else`
- `elif` `else`
- Python
-

3.1.4 if

`if`

```
# login_system.py
username = input("        ")
password = input("        ")

if username == "admin":
    if password == "123456":
        print("        ~")
        role = "admin"
    elif password == "admin123":
        print("        ")
    else:
        print("        ")
else:
    print("        ")
```

Web

and

```
if username == "admin" and password == "123456":
    print("        ")
```

3.2

Python

while

for...in

3.2.1 while

```
while True:
    #
    # False
```

1 100

```
# sum_1_to_100.py
total = 0
i = 1

while i <= 100:
    total += i
    i += 1

print("1 100 ", total)
```

1 100 5050

while + if


```
# guess_number.py
import random

target = random.randint(1, 100)
count = 0
print("          1-100          ")

while True:
    guess = int(input(" "))
    count += 1

    if guess > target:
        print(" ~")
    elif guess < target:
        print(" ")
    else:
        print(f"          {count} ")
        break
```

while if-elif-else break

while True break “ ”

3.2.2 for...in

for

```
for in :
    #
```

1 range()

range(start, stop, step)

```
# for_range.py
print("    0    9 ")
for i in range(10):          # 0~9
    print(i, end=' ')

print("\n\n    5    15    ")
for i in range(5, 16, 2):
    print(i, end=' ')
```

2

```
# for_iterate.py
fruits = ["apple", "orange", "banana", "grape"]
```

```
for fruit in fruits:
    print(fruit)
```

```
for char in "Python":
    print(char)
```

```
person = {"name": "  ", "age": 28, "city": "  "}
for key, value in person.items():
    print(f"{key}: {value}")
```

```
# multiplication_table.py
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f"{j}×{i}={i*j}", end="\t")
    print() #
```

“ ” “ ”

3.3 break continue pass

3.3.1 break

```
# break_example.py
numbers = [3, 7, 12, 15, 22, 28]

for num in numbers:
    if num % 2 == 0 and num % 7 == 0:
        print(f"      2 7      {num}")
        break
    else:
        print("      ")
```

Python `for` `while` `else` `break` `else`

Python

3.3.2 continue

```
# continue_example.py
for i in range(1, 11):
    if i % 2 == 1: #
        continue
    print(f"  {i}")
```

2 4 6 8 10

ERROR

3.3.3 pass

`pass`

```
# pass_example.py
for i in range(5):
    if i == 2:
        pass    #
    else:
        print(i)
```

pass “ ”

3.4

1

```
import random

print("===          ===")
target = random.randint(1, 100)
attempts = 0
max_attempts = 10

while attempts < max_attempts:
    try:
        guess = int(input(f" {attempts+1}          1-100  "))
    except ValueError:
        print("          ")
        continue

    attempts += 1

    if guess < 1 or guess > 100:
        print("          1-100  ")
        continue
    if guess > target:
        print("          ")
    elif guess < target:
        print("          ")
    else:
        print(f"          {attempts}          ")
        break
else:
    print(f"          {max_attempts}          {target} ")

print("          ")
```

`while` `if-elif-else` `break` `continue` `try-except`

`else`

`while + if`

`for +`

3.5

- `if-elif-else` " "
- `while` " " `for` " "
- `break` " " `continue` " " `pass` " "
-
-

1. if-elif-else 2. while 1-100
3 5 3. for range() 1 1000 4. "
" continue 5. 6. ATM
1. 2. 3. 4. while + if + break 7. [88, 65,
92, 45, 78] 8. 1-9
" "

5200

2000

Python 3.10+

"

"

4

Python

4

.....

bug

Function

- - - -

1.

2.

3.

4.

Linux

Shell

LEGB

lambda

4.1

4.1.1

Python `def`

```
def ( ):  
    """ docstring """  
    #  
    return # return None
```

- snake_case calculate_average jq
- : 4
-
-

```
def greet(name=" ", times=1):
    """
    :param name:
    :param times: 1
    """
    for _ in range(times):
        print(f" {name} Python ")
    print("--- ---")
```

```
greet() #
greet(" ") #
greet(" ", 3) #
greet(times=2, name=" ") #
```

```

Python
--- ---
Python
--- ---
Python
Python
Python
--- ---
Python
Python
--- ---
```

docstring

help(greet) IDE

4.2

Python

4.2.1 Positional Arguments

```
def rectangle_area(width, height):  
    """  
    """  
    area = width * height  
    print(f" {width} {height} {area}")  
    return area  
  
rectangle_area(5, 8)    # width=5, height=8
```

4.2.2 Default Arguments

```
def power(base, exponent=2):  
    """  
    """  
    return base ** exponent  
  
print(power(3))          # 9  
print(power(3, 4))       # 81  
print(power(exponent=5, base=2)) #
```

```
def bad_append(item, items=[]): #
    items.append(item)
    return items
```

```
print(bad_append(1)) # [1]
print(bad_append(2)) # [1, 2] ←
```

```
def good_append(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

```
print(good_append(1)) # [1]
print(good_append(2)) # [2] ←
```

list dict

Python

4.2.3 Keyword Arguments

```
def student_info(name, age, city=" "):
    print(f"      {name}      {age}      {city}")

student_info(age=18, name=" ", city=" ") #
```

4.2.4 *args* *kwargs

- `*args`
- `kwargs`

```
def team_info(members, *info):  
    """  
    """  
    print(" ")  
    for member in members:  
        print(f" - {member}")  
  
    print("\n ")  
    for key, value in info.items():  
        print(f" {key}: {value}")  
  
team_info(" ", " ", " ", " ",  
         project=" ",  
         budget=500000,  
         leader=" ")
```

-> -> *args* -> -> *kwargs

```
def complex_func(a, b=2, args, c=10, *kwargs):  
    print(a, b, args, c, kwargs)  
  
complex_func(1, 3, 4, 5, 6, c=99, name="test", debug=True)
```

4.3

—— `return`

```
def divide(x, y):
    """
    """
    if y == 0:
        print("      0 ")
        return None          #      None
    return x / y             #

result1 = divide(10, 2)
result2 = divide(10, 0)
print("10/2 =", result1)
print("10/0 =", result2)
```

Python

```
def get_student_score(name):
    """
    """
    scores = [85, 92, 78]
    total = sum(scores)
    avg = total / len(scores)
    return name, total, round(avg, 1)  #

name, total, avg = get_student_score(" ")
print(f"{name}    {total}    {avg}")
```

`return`

`None`

4.4

LEGB

Python **LEGB**

- **L**ocal
- **E**nclosing /
- **G**lobal
- **B**uilt-in Python / `len` `int`

```
x = 100 #

def outer():
    x = 50 # Enclosing

    def inner():
        x = 10 # Local
        print("inner x:", x)

    inner()
    print("outer x:", x)

outer()
print("global x:", x)
```

```
inner x: 10
outer x: 50
global x: 100
```

`global` `nonlocal`

```
counter = 0

def increment():
    global counter    #
    counter += 1
    print("    ", counter)

increment()
increment()
print("    ", counter)
```

nonlocal

```
def outer_counter():
    count = 0

    def inner_counter():
        nonlocal count    #
        count += 1
        return count

    return inner_counter

my_counter = outer_counter()
print(my_counter())    # 1
print(my_counter())    # 2
```

global

4.5

Closure

```
def make_multiplier(factor):  
    """    """  
    def multiplier(x):  
        return x * factor    #    factor  
    return multiplier  
  
double = make_multiplier(2)  
triple = make_multiplier(3)  
  
print(double(5))    # 10  
print(triple(5))    # 15
```

decorator

4.6 Python

Python import

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
print("    :", len(numbers))  
print("    :", max(numbers))  
print("    :", min(numbers))  
print("    :", sum(numbers))  
print("    :", sorted(numbers))  
print("    :", round(3.14159, 2))
```

enumerate

```
for index, value in enumerate(numbers, start=1):  
    print(f" {index}    {value}")
```

zip

```
names = ["  ", "  ", "  "]
ages = [18, 20, 22]
for name, age in zip(names, ages):
    print(f"{name} {age} ")
```

map

```
squares = list(map(lambda x: x2, numbers))  
print("      :", squares)
```

filter

```
evens = list(filter(lambda x: x % 2 == 0, numbers))  
print("      :", evens)
```

`abs()` `int()` `float()` `str()` `type()` `isinstance()`
`range()` `input()` `print()` `help()` `dir()` <https://docs.python.org/zh-cn/3/library/functions.html>

4.7 lambda

`def` `lambda`

`lambda` `:`

```
#         vs lambda
def add(x, y):
    return x + y

add_lambda = lambda x, y: x + y

print(add(3, 5))          # 8
print(add_lambda(3, 5))  # 8
```

1 sorted

```
students = [(" ", 85), (" ", 92), (" ", 78)]
students_sorted = sorted(students, key=lambda s: s[1], reverse=True)
print(" ", students_sorted)
```

2 map filter

```
data = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x*x, data))
print("  :", squared)
```

lambda

- `sorted` `map` `filter` `max` `key`
-

lambda

def

4.8

```

def add_student(students, name, scores):
    """
    """
    if name in students:
        print(f"{name} already exists")
    students[name] = scores
    print(f"Added student {name} with scores {scores}")

def calculate_average(scores):
    """
    """
    if not scores:
        return 0
    return sum(scores) / len(scores)

def get_class_average(students):
    """
    """
    if not students:
        return 0
    total_avg = sum(calculate_average(scores) for scores in students.values())
    return total_avg / len(students)

def print_top_student(students):
    """
    """
    if not students:
        print("No students")
        return
    top = max(students.items(), key=lambda item: sum(item[1]))
    name, scores = top
    print(f"Top student: {name} with scores {sum(scores)} and average {calculate_average(scores):.1f}")

def main():
    students = {}

    add_student(students, "Alice", [85, 90, 78])

```



```

add_student(students, " ", [92, 88, 95])
add_student(students, " ", [76, 82, 80])

print("\n===      ===")
for name, scores in students.items():
    print(f"{name}:      {scores}      {calculate_average(scores):.1f}")

print(f"\n      {get_class_average(students):.2f}")
print_top_student(students)

if __name__ == "__main__":
    main()

```

lambda

sum max

4.9

1. 2. verb_noun get_user_info 3.
- 5-6 4. 5. 6. **global**
7. 50

-
-
- lambda lambda nonlocal

Python

lambda

1.

`*args`

2. LEGB

3. `lambda sorted`

4.

5. `make_power(n)`

n

6500

Python

5 List

Python 5 “
”
“ ”

List

Python
Linux “ ” Python

-
-
-
-
-

Python

5.1

[]

- 1.
2. **Mutable**

```
# 1
shopping_list = [' ', ' ', ' ', 3.5, True]
print(" ", shopping_list)
```

2 list()

```
name_str = "Python"
name_list = list(name_str)
print(" ", name_list)
```

```
empty_list1 = []  
empty_list2 = list()  
print("    1    ", len(empty_list1))  
print("    2    ", len(empty_list2))
```

```
mixed_list = [42, "Hello", [1, 2, 3], True, 3.14159]
print("      ", mixed_list)
```

```
[' ', ' ', ' ', ' ', ' ', 3.5, True]
['P', 'y', 't', 'h', 'o', 'n']
1    0
2    0
[42, 'Hello', [1, 2, 3], True, 3.14159]
```

Python

Excel

API

Set

5.2

Index

Python

0

5.2.1

```
fruits = ['apple', 'banana', 'cherry', 'date', 'elderberry']

print(" 1      0 ", fruits[0])
print(" 3      2 ", fruits[2])
print("      -1 ", fruits[-1])
print(" 2      -2 ", fruits[-2])
```

```
fruits[0] = 'apple'
print("      ", fruits)
```

Slicing Python

[: :]

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(" 3      ", numbers[0:3])
print(" 2      ", numbers[2:])
print(" 6      6 ", numbers[:6])
print("      ", numbers[:])
print(" 2      ", numbers[::2])
print("      ", numbers[::-1]) # -1
```

```
def safe_get(lst, index):  
    try:  
        return lst[index]  
    except IndexError:  
        return "IndexError: list index out of range" + str(len(lst))  
  
scores = [85, 92, 78, 96, 88]  
print(safe_get(scores, 2))  
print(safe_get(scores, 10)) #
```

`data[0:10]`

`10`

`data[::-1]`

5.3 CRUD

5.3.1 Create & Add

```
tasks = [' ', ' ']
```

append()

```
tasks.append(' ')\nprint("append ", tasks)
```

insert()

```
tasks.insert(1, '  ')\nprint("insert ", tasks)
```

extend()

```
more_tasks = [' ', ' ']  
tasks.extend(more_tasks)  
print("extend ", tasks)  
print("      ", len(tasks))
```

5.3.2 Delete

```
heroes = [' ', ' ', ' ', ' ', ' ', ' ']
```

remove()

```
heroes.remove(' ')\nprint("remove ", heroes)
```

pop()

```
last = heroes.pop()  
print("pop      ", last)  
print("pop     ", heroes)
```

del clear()

```
del heroes[0]            #  
print("del  ", heroes)  
heroes.clear()          #  
print("clear  ", heroes)
```


5.3.3 Update & Read

```
data = [10, 20, 30, 20, 40]
```

```
data[1] = 25  
data[2:4] = [35, 45]      #  
print("    ", data)
```

```
print("20      ", data.count(20))
print("35      ", data.index(35))
print("50      ", 50 in data)
print("      ", len(data))
```

```
append O(1)      insert(0, value) O(n)
collections.deque
```

5.4

```
cities = [' ', ' ', ' ', ' ']
```

1

```
for city in cities:  
    print(" ", city)
```

2

enumerate

```
for i, city in enumerate(cities, 1):    # 1    1
    print(f" {i}          {city}")
```

3 while

```
i = 0
while i < len(cities):
    print("while ", cities[i])
    i += 1
```

zip

```
scores = [85, 92, 78, 96]
for city, score in zip(cities, scores):
    print(f"{city}      {score}")
```

5.5

```
scores = [87, 65, 92, 78, 91, 83]
```

1. `sort()`

```
scores.sort()                                #  
print("      ", scores)                     #  
scores.sort(reverse=True)                   #  
print("      ", scores)
```

2. sorted()

```
original = [87, 65, 92, 78]
new_sorted = sorted(original, reverse=True)
print("sorted      ", original)
print("          ", new_sorted)
```

```
words = ['apple', 'hi', 'banana', 'a']  
words.sort(key=len)  
print("      ", words)
```

```
numbers = [1, 2, 3, 4]
numbers.reverse()
print(" ", numbers)
```

```
key=lambda x: x['score']
```

5.6 Python “ ”

for

```
#  
squares = [x2 for x in range(1, 11)]  
print("1 10      ", squares)
```

```
even_squares = [x2 for x in range(1, 21) if x % 2 == 0]  
print("          ", even_squares)
```

```
raw_data = ['Python', '', 'JAVA', 'Go', ' ']  
cleaned = [item.strip().lower() for item in raw_data if item.strip()]  
print("      ", cleaned)
```

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
print("      ", flat)
```

```
for
for +append()
for
```

5.7

```
# student_score_system.py
students = [
    {"name": " ", "math": 85, "english": 92, "chinese": 78},
    {"name": " ", "math": 96, "english": 88, "chinese": 95},
    {"name": " ", "math": 72, "english": 80, "chinese": 68},
    {"name": " ", "math": 91, "english": 85, "chinese": 89}
]
```

1.

```
for student in students:
    total = student["math"] + student["english"] + student["chinese"]
    student["total"] = total
    student["avg"] = round(total / 3, 1)
```


2.

```
sorted_students = sorted(students, key=lambda x: x["total"], reverse=True)

print("===      ===")
for rank, s in enumerate(sorted_students, 1):
    print(f" {rank}  {s['name']}    {s['total']}    {s['avg']}")
```

3.

```
failed_math = [s["name"] for s in students if s["math"] < 60]
print(" ", failed_math if failed_math else " ")
```

4.

```
math_avg = sum(s["math"] for s in students) / len(students)
print(f"          {round(math_avg, 2)}")
```

CSV

5.8

```
import copy

a = [1, 2, [3, 4]]
b = a.copy()      #
c = copy.deepcopy(a) #

a[2][0] = 999      #

print("a =", a)
print("b (  ) =", b)  #
print("c (  ) =", c)  #
```

1.

for item in lst[:]: 2.

3.

- `lst[:]` `copy.deepcopy()`
- `pandas`
-

5.9

Python

CRUD

1. 5
4, 4, 5]

TODO

max/min 2.
set 3.
+ 5.

[1, 2, 2, 3,
4.

Dict

4200

Python 3.8+

Linux

“ ”

Python

6

Python 6

Python " "

" " " " bug

Linux

6.1 Tuple — " "

6.1.1 " "

tuple ()

python

```

empty_tuple = () #      single_tuple = (5,) #      5 colors =
('red', 'green', 'blue') point = 10, 20, 30 #      Python

print(type(empty_tuple)) # print(single_tuple) # (5,) print(colors[1]) #
green

```

(5)

type((5))

Python

Python

1.

2. hashable

3.

4.

key

bug

key

(800, 600)

6.1.2

Packing

Unpacking —

“ ”

“ ”

python

coordinates = 39.9, 116.3, " " #

```
latitude, longitude, city = coordinates print(f"    {latitude}  
{longitude}    {city}")
```


39.9

116.3

```
a, b = 10, 20 a, b = b, a print(a, b) # 20 10
```

```
*
```

```
python numbers = (1, 2, 3, 4, 5) first, second, *rest = numbers print(first,  
second) # 1 2 print(rest) # [3, 4, 5]    rest
```

```
def get_student_info(): return " ", 18, " ", (" ", " ", " ")
```

```
name, age, grade, *subjects = get_student_info() print(f"{name} {age} {grade} {subjects}")
```

-
- `"%s %d" % (" ", 18)`
- `enumerate()` for `i, value in enumerate(lst):`
- `RGB (255, 0, 0)`

6.1.3

- `count(value)`
- `index(value)`

```
python scores = (85, 92, 88, 92, 95, 92) print(scores.count(92)) # 3  
print(scores.index(88)) # 2
```

```
score_list = list(scores) score_tuple = tuple(score_list)
print(type(score_list), type(score_tuple))
```

```
python cities = ( (" ", 39.9, 116.3), (" ", 31.2, 121.5), (" ", 23.1,
113.3) )
```

```
for city, lat, lon in cities: print(f"{city}      ({lat}, {lon})")
```

-

-

6.2 Dictionary — “ ”

key value

Hash Table Python key

0(1)

6.2.1

python

```
empty_dict = {} student = {"name": " ", "age": 18, "score": 95} person
= dict(name=" ", age=45, city=" ")

print(student["name"]) #
```

/

```
student["gender"] = " " #    student["score"] = 98 #
```

get KeyError

```
print(student.get("score")) # 98 print(student.get("phone", "  ")) #
```

```
student.pop("gender") #  
student.clear() #
```

```
del student["age"] #
```

```
setdefault
```

```
key
```

```
python from collections import defaultdict
```

```
word_count = {} word_count.setdefault("Python", 0)
word_count["Python"] += 1
```

```
count = defaultdict(int) count["Python"] += 5 print(count["Python"]) # 5
```

```
6.2.2
```

```
python student = {"name": " ", "age": 17, "score": 92, "city": " "}
```

key

```
for key in student: print(key, student[key])
```

items()

```
for key, value in student.items(): print(f"{key}: {value}")
```

```
for value in student.values(): print(value)
```



python

```
names = ["  ", "  ", "  "] scores = [85, 92, 78] score_dict = {name: score  
for name, score in zip(names, scores)} print(score_dict)
```

{' ': 85, ' ': 92, ' ': 78}



```
squared = {x: x2 for x in range(6) if x % 2 == 0} print(squared) # {0: 0, 2: 4, 4: 16}
```

6.2.3

1. JSON
- 2.
3. memoization
4. API

```
python students = { " ": {"math": 92, "english": 88, "chinese": 95}, " ": {"math": 85, "english": 96, "chinese": 90}, " ": {"math": 78, "english": 82, "chinese": 88} }
```

```
def print_average(): for name, scores in students.items(): avg = sum(scores.values()) / len(scores) print(f"{name} {avg:.1f} ")
```

```
print_average()
```

Python 3.6

`collections.OrderedDict`

6.3 Set —

6.3.1

python

```
empty_set = set() # {}          fruits = {"apple", "banana",  
"apple", "orange"} #          print(fruits) # {'apple', 'banana', 'orange'}
```

```
visitors = [" ", " ", " ", " ", " ", " "] unique_visitors = set(visitors)
print(len(unique_visitors)) # 3
```

- `add()`
- `remove()`
- `discard()`
- `pop()`

6.3.2

```
python python_members = {" ", " ", " ", " ", " "} java_members = {" ",
" ", " ", " ", " "}
```

```
all_members = python_members | java_members print(all_members)
```

```
common = python_members & java_members print(common) # {' ', '}'
```

python java

```
only_python = python_members - java_members  
print(only_python)
```

```
sym_diff = python_members ^ java_members
print(sym_diff)
```

```
union() intersection() difference() symmetric_difference()
```

- `issubset()`
- `isdisjoint()`

6.3.3 frozenset

key	frozenset
-----	-----------

```
python frozen_tags = frozenset([" ", " ", "Python"])
info = {"frozen_tags":
        " "}
```


1.

2.

3.

4. UV

—

```
python user1_hobbies = {" ", " ", " ", " ", " ", " "} user2_hobbies = {" ", " ", " ", " ", " ", " "}
```

```
common_hobbies = user1_hobbies & user2_hobbies print(f"
{common_hobbies}") print(f"      {len(common_hobbies) /
len(user1_hobbies | user2_hobbies):.2%}")
```

6.4

list							
tuple							dict key
dict			(3.6+)	key			
set							

- → dict
- “ ” → set
- → tuple
- → list

6.5

tuple dict set

python

tuple

```
CLUB_INFO = (" ", " ", " ", " ")
```

dict

```
students = { " ": {"clubs": {" " ", " "}, "score": 92}, " ": {"clubs": {" " ", " "}, "score": 88}, " ": {"clubs": {" " "}, "score": 95} }
```

```
def find_common_members(club1, club2): """" """" members1  
= {student for student, info in students.items() if club1 in info["clubs"]}  
members2 = {student for student, info in students.items() if club2 in  
info["clubs"]} return members1 & members2
```

```
print(" ", find_common_members(" ", " "))
```

set

```
club_count = {} for info in students.values(): for club in info["clubs"]:
club_count.setdefault(club, 0) club_count[club] += 1

print("      ", club_count)
```

6.6

Python

```
1.
2.
3.
4.      1~10
5.          dict
6.
7.          key
8.      "      " set      "      "
```

5200

6500

Python 3.8+

	Linux	---	"	""	""
"	2000	Markdown			

Python 7

#	7				
	Python	7			
Linux		grep sed awk		Linux	
API					
7.1					
Python	'	"		")	

python

```
name = " " greeting = 'Hello, Python!' multiline = """  
print(type(name)) # print(multiline)
```

7.1.1 Immutability

“ ”

```
python s = "hello" print(" :", id(s))  
s2 = s.replace("h", "H") # print(" :", id(s2)) print("  
:", s) # "hello"
```

Python

1. intern
2. key
3. + join()

7.1.2

```
python text = "Python" print(" :", len(text)) # 11 1
print(" :", text[0]) # P print(" :", text[-1]) # print(" 1:",
text[0:6]) # Python print(" 2:", text[6:]) # print(" :", text[::-1])
```

Tips len()

7.2

Python

7.2.1

python

```
log = "2026-08-19 14:30:25 [ERROR] [user:admin] Login failed from  
192.168.1.100"
```

```
print("ERROR :", log.find("[ERROR]")) #      print("      admin:",  
log.rfind("admin")) print("ERROR      :", log.count("ERROR")) print("  
:", log.startswith("2026")) print("      IP:", "192.168." in log)
```

```
print("      :", "12345".isdigit()) # True print("      :",  
"user123".isalnum()) # True
```

- `find()` `-1`
- `index()`
- `URL` `startswith()` `endswith()`

7.2.2

python

```
content = "    Java Java"    " print(content.replace("Java", "Python"))
```

```
print(content.replace("Java", "Python", 1))
```

```
user_input = "  \n" clean_user = user_input.strip() #
print(repr(clean_user)) # ' '

print(" left only ".lstrip()) #      print(" right only ".rstrip()) #
```

```
Windows \   Linux /
```

7.2.3

```
split()  join()      "      "
```

python

CSV

```
data_line = " ,25, , ,13800138000" fields = data_line.split(",") print("
:", fields[0]) print(" :", fields[1]) print(" :", fields[2]) print(" :",
fields[3])
```

join()

```
user_list = [" ", "30", " ", " "] csv_row = ",".join(user_list) print(" CSV  
:", csv_row)
```

```
print(" | ".join(user_list))
```

```
split()
```

```
python
```

```
messy_text = " hello world \t python " words = messy_text.split() #  
print(words) # ['hello', 'world', 'python']
```

```
multi_line = "" "" print(multi_line.splitlines())
```

```
s = s + new_part
```

```
''.join(lis
```

python

```
result = "" for i in range(10000): result += str(i) #
```

```
parts = [] for i in range(10000): parts.append(str(i)) result = "".join(parts)
```

7.3

Python

7.3.1 %

```
python name = " " age = 35 score = 87.456
```

```
print(" %s %d %.2f " % (name, age, score)) print(" %.1f%%" % 88.7) # %
```

7.3.2 str.format()

```
python print(" {0} {1} {2:.2f} ".format(name, age, score))
```

```
print("    {name}    {city}".format(name="  ", city="  "))
```

```
print("{:<10} | {:>8} | {:^10}".format(" ", " ", " "))
```

7.3.3 f-string Python 3.6+

python

```
name = " " age = 28 score = 92.75 level = " "
```

```
print(f" {name} {age} {score:.1f} {level}")
```

```
numbers = [1, 2, 3, 4, 5] print(f"    {len(numbers)}  
{sum(numbers)/len(numbers):.2f}")
```

- Python 2/3.5 `format()`
- Python 3.6+ **f-string**
- `format()`

7.4

str bytes

- str Unicode
- bytes `b''`

python

```
text = " Python " # str    byte_data = text.encode("utf-8") #    bytes
print(type(text)) # print(type(byte_data)) # print(byte_data) #
b'\xe4\xbd\xa0\xe5\xa5\xbd\xef\xbc\x8cPython\xef\xbc\x81'
```

```
recovered = byte_data.decode("utf-8") print(recovered) #    Python
```

7.4.1

- **UTF-8** UTF-8
- **GBK/GB2312** Windows
- **ASCII**

```
UnicodeDecodeError: 'utf-8' codec can't decode byte...
```

python

```
with open("data.txt", "r", encoding="utf-8", errors="replace") as f:  
content = f.read()
```

errors

strict

ignore

replace ?



python

```
def read_file_safely(filename): for encoding in ["utf-8", "gbk", "gb2312",  
"ansi"]: try: with open(filename, "r", encoding=encoding) as f: return  
f.read(), encoding except UnicodeDecodeError: continue raise  
ValueError(" ")
```

```
content, used_encoding =  
read_file_safely("log.txt")
```

```
print(f"    {used_encoding}  
    ")
```

API bytes .decode()

7.5

1

python

log_analyzer.py -

```
logs = [ "2026-08-19 10:00:01 INFO User login success: admin",  
"2026-08-19 10:01:23 ERROR Database connection failed", "2026-08-19  
10:05:45 WARNING Disk space low", "2026-08-19 10:10:12 ERROR User  
login failed: guest" ]
```

```
error_count = 0 error_messages = []
```

```
for line in logs: if "ERROR" in line: error_count += 1 #         parts =  
line.split("ERROR") error_messages.append(parts[1].strip())
```

```
print(f"         {error_count}         ") for msg in error_messages: print(" -",  
msg)
```

```
2         f-string + join
```

python

```
raw_data = [ "    , 25 ,    ,    " , "    , 30 ,    ,    " , "    , 28 ,    ,    " ]
```

```
cleaned = [] for line in raw_data: fields = [field.strip() for field in  
line.split(",")] cleaned.append(fields)
```

```
print("===          ===") print("{:<6} {:<4} {:<8} {:<10}".format("  ",
"  ", "  ", "  ")) print("-" * 35) for row in cleaned: print(f"{row[0]:<6}
{row[1]:<4} {row[2]:<8} {row[3]:<10}")
```

7.6

```
1.          +          + join()
2.          strip()
3.          encoding="utf-8"
4.          re
5.          UTF-8      Windows Linux
```

7.7

Python

find/replace/split/join f-string

```
1.
2.          IP          split()
3.          "{name}{place}"          format()    f-string
4.          "2026-08-19"          "20260819"
```

4500

2000

8 I/O

Python

8

Python

Python

“ ” “ ”

- .log
- JSON
- CSV
-
-
- CMS

Linux

“ ”

8.1 open ()

8.1.1

Python

“ ”

8.1.2 open()

`open()`

python `file_object = open(file, mode='r', encoding=None, errors=None)`

- file
- mode
- encoding `encoding='utf-8'`

					encoding
'r'					
'w'					
'a'					
'r+'					
'rb'					exe
'wb'					
Windows		encoding='utf-8'		GBK	UTF-8

8.1.3

1

python

file_demo.py

```
def main(): # 1.      with open('demo.txt', 'w', encoding='utf-8') as f:
f.write("      Python      \n") f.write("      \n") f.writelines(["      \n", "
    writelines\n"]) print("      ") # 2.      with open('demo.txt', 'r',
encoding='utf-8') as f: print("===      read()      ===")
print(repr(f.read())) # repr()      f.seek(0) #
print("\n===      readlines()      ===") lines = f.readlines() for i, line in
enumerate(lines, 1): print(f"{i}: {repr(line)}")

if __name__ == "__main__": main()
```

```
===      read()      === '      Python      \n      \n      \n
writelines\n'

===      readlines()      === 1: '      Python      \n' 2: '      \n' ...
```

read() readline() readlines()

- read()
- readline()
- readlines()

close()

close()

python with open('source.jpg', 'rb') as src: data = src.read() with
open('backup.jpg', 'wb') as dst: dst.write(data) print(" ")

8.2 with

`open() + close()`

if for try

Python

with

8.2.1 with

with

Python

`__exit__`

- 1.
- 2.
- 3.

```
python with open('config.json', 'r', encoding='utf-8') as f: content =  
f.read()
```

f

```
python with open('source.txt', 'r', encoding='utf-8') as src, \
open('target.txt', 'w', encoding='utf-8') as dst: dst.write(src.read()) print("
")
```

99% with

8.3

Windows \ Linux/Mac /

8.3.1 vs

- data/config.json
- C:\project\data\config.json /home/user/project/data/config.json

pathlib Python 3.4+

```
python from pathlib import Path
```

```
current_dir = Path(__file__).parent data_dir = current_dir / "data"
data_dir.mkdir(exist_ok=True) #
```

```
config_file = data_dir / "config.json" print(config_file) #
```

```
if config_file.exists(): print("      ")
```



os

```
python import os
```

```
base_dir = os.path.dirname(os.path.abspath(__file__)) data_path =
os.path.join(base_dir, "data", "config.json") print(data_path)
```

pathlib

8.4 JSON

JSON

API

NoSQL

8.4.1 json

- `json.dump(obj, fp)` Python
- `json.load(fp)` JSON Python
- `json.dumps(obj)` JSON
- `json.loads(s)` Python

1

```
python import json from pathlib import Path
```

```
def save_config(): config = { "user": "  ", "theme": "dark", "font_size": 14,
"favorite_languages": ["Python", "JavaScript"], "settings": { "auto_save":
True, "backup_interval": 30 } } config_path = Path("config.json") with
open(config_path, 'w', encoding='utf-8') as f: json.dump(config, f,
ensure_ascii=False, indent=4) print("      ")
```

```
def load_config(): config_path = Path("config.json") if not
config_path.exists(): print("      ") save_config() with
```

```

open(config_path, 'r', encoding='utf-8') as f: config = json.load(f) print("
:", config["user"]) print("  :", config["theme"]) print("    :",
config["favorite_languages"]) return config

if __name__ == "__main__": save_config() load_config()

```

- ensure_ascii=False
- indent=4 JSON
-

8.5 CSV

CSV Comma-Separated Values

8.5.1 csv

DictReader DictWriter

2

```
python import csv from pathlib import Path from datetime import datetime
```

```

def export_scores(): scores = [ {" " " " ", " " ": 95, " " ": 88, " " ": 183},
{" " " " ", " " ": 92, " " ": 96, " " ": 188}, {" " " " ", " " ": 85, " " ": 78, "

```

```
": 163}], ] csv_path = Path("scores.csv") with open(csv_path, 'w',
encoding='utf-8-sig', newline='') as f: # utf-8-sig Excel writer =
csv.DictWriter(f, fieldnames=["name", "score", "total", "count"]) writer.writeheader() #
writer.writerows(scores) print("done")
```

```
def analyze_scores(): csv_path = Path("scores.csv") with open(csv_path, 'r',
encoding='utf-8-sig') as f: reader = csv.DictReader(f) total = 0 count = 0
for row in reader: score = int(row["score"]) total += score count += 1
print(f"{row['name']} {score} ") print(f"\n total : {total/count:.1f} ")
```

```
if __name__ == "__main__": export_scores() analyze_scores()
```

- encoding='utf-8-sig' BOM Excel
- utf-8-sig

8.6

```
python from pathlib import Path
```

```
def safe_read(file_path): try: with open(file_path, 'r', encoding='utf-8') as
f: return f.read() except FileNotFoundError: print(f"File not found: {file_path}")
return None except PermissionError: print("Permission denied") return None
except Exception as e: print(f"Error: {e}") return None
```



```

1.         with
2.             encoding='utf-8'    utf-8-sig
3.         for line in f:
4.             pathlib
5.                 f.flush()
6.

```

“ ”

- JSON
- CSV
- “ ” CSV

8.7

-
- with
- pathlib
- JSON
- CSV DictReader/Writer

“ ”

1. JSON
2. CSV
3. logs pathlib datetime
4. JSON

5200

- Python open() json csv pathlib
-

9

Python

Python

9

“ ”

@.....

“ ”—

Exception Handling

bug “ ” Debugging

“

9.1

Error

Exception

- **SyntaxError** “ ” Python
- **Exception** Python

Python

BaseException

Exception

Python

Traceback

- 1.
- 2.
- 3.

Traceback

Python

9.2 Python

Python

10

python

1: ZeroDivisionError -

```
def divide(a, b): return a / b
```

```
try: result = divide(10, 0) except Exception as e: print(type(e).__name__,  
":", e)
```

: ZeroDivisionError : division by zero



python

2: IndexError -

```
fruits = ['apple', 'banana'] print(fruits[5]) # IndexError: list index out of range
```



python

3: KeyError -

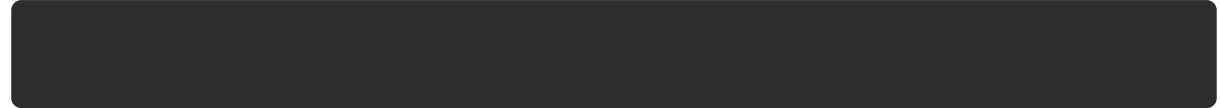
```
person = {'name': ' ', 'age': 18} print(person['salary']) # KeyError:  
'salary'
```



python

4: TypeError -

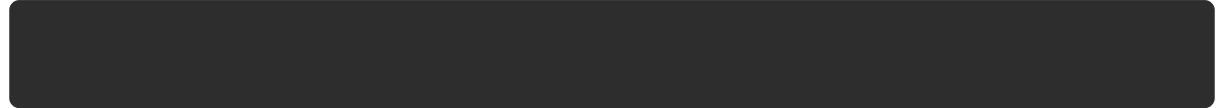
```
print("hello" + 5) # TypeError: can only concatenate str (not "int") to str
```



python

5: ValueError -

```
print(int("abc")) # ValueError: invalid literal for int() with base 10: 'abc'
```



python

6: FileNotFoundError -

```
with open('      .txt', 'r') as f: content = f.read()
```



python

7: NameError -

```
print(undefined_variable) # NameError: name 'undefined_variable' is not defined
```



python

8: AttributeError -

```
num = 123 print(num.append(4)) # AttributeError: 'int' object has no  
attribute 'append'
```

python

9: ImportError / ModuleNotFoundError

```
import non_existing_module # ModuleNotFoundError
```



python

10: OSError

PermissionError

80% FileNotFoundError ValueError TypeError KeyError

9.3 try-except

try...except

python

```
def safe_divide(x, y):
    try:
        result = x / y
        print(f"Result: {result}")
    except ZeroDivisionError:
        print("Cannot divide by 0")
    except TypeError:
        print("Invalid input")
    except Exception as e:
        print(f"Exception: {e}")
```

```
safe_divide(10, 2)
safe_divide(10, 0)
safe_divide(10, "a")
```

```
try:
    # Code that might raise an exception
except Exception as e:
    # Handle the exception
    bug_report = f"An exception occurred: {str(e)}"
    print(bug_report)
```

```
python try:
    num = int(input("Enter a number: "))
    print(100 / num)
except (ValueError, ZeroDivisionError) as e:
    print(f"Error: {str(e)}")
```

9.4 try-except-else-finally

```
python try: #          pass except Exception1: #          pass except
Exception2 as e: #          pass else: #          pass finally: #
          pass
```

```
1.    try
2.    try          else          finally
3.    try          except          finally
4.          except          finally
```

python

complete_file_handler.py

```
import os
```

```
def read_file_safely(filename): content = None try: print(f"      :  
{filename}") f = open(filename, 'r', encoding='utf-8') content = f.read()  
print("      ") #      number = int(content.strip()) except  
FileNotFoundError: print(f"      '{filename}'      ") except  
PermissionError: print("      ") except ValueError as e: print("  
      :", e) except Exception as e: print("      :", type(e).__name__,  
str(e)) else: print("      :", number) return number finally:  
print("finally      ") if 'f' in locals() and not f.closed: f.close() print("  
") return content
```

```
with open('test.txt', 'w', encoding='utf-8') as f: f.write("42")
```

```
print("===      ===") result = read_file_safely('test.txt') print(":" , result)
```

```
print("\n===      ===") read_file_safely('ghost.txt')
```

- `finally`
- Python `with` `try-finally` `finally`
- `finally`

9.5 `raise`

“ ”

```
python def set_age(age): if not isinstance(age, int): raise TypeError("
") if age < 0 or age > 150: raise ValueError(f" {age} ") print(f"
: {age} ")
```

```
try: set_age(-5) except ValueError as e: print("    :", e)
```



```
python class InvalidEmailError(Exception): """          """ def
__init__(self, email, message="    "): self.email = email self.message =
f"{message}: {email}" super().__init__(self.message)
```

```
def validate_email(email): if '@' not in email or '.' not in email: raise
InvalidEmailError(email) print("    ")
```

```
try: validate_email("invalid-email-com") except InvalidEmailError as e:
print("    :", e)
```

9.6

bug " " " "

6.1 print()

print

6.2 assert

```
python def calculate_average(numbers): assert len(numbers) > 0, "  
    " assert all(isinstance(x, (int, float)) for x in numbers), "  
    " return  
sum(numbers) / len(numbers)
```

```
try: print(calculate_average([])) except AssertionError as e: print(" :", e)
```

assert python -0

6.3 pdb

bash

```
import pdb pdb.set_trace() #
```

```
n(next) c(continue) l(list) p variable( ) q(quit)
```

6.4 IDE VS Code / PyCharm

VS Code

1. “ ”
- 2.
3. F5
- 4.

print assert

IDE pdb

9.7 logging print

print

logging

```
python import logging
```

```
logging.basicConfig( level=logging.DEBUG, #          format='%  
(asctime)s - %(name)s - %(levelname)s - %(message)s',  
filename='app.log', #          filemode='a' #          )
```

```
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)
logging.getLogger().addHandler(console_handler)
```

```
logging.debug("                ") logging.info("                ")
logging.warning("            ") logging.error("            ") logging.critical("            ")
                "
```

```
def process_data(data): if not data: logging.error(" ") return None
logging.info(f" {len(data)} ") return [x * 2 for x in data]
```

```
DEBUG < INFO < WARNING < ERROR < CRITICAL
```

- level=DEBUG
- level=INFO WARNING

9.8

```
python import logging import sys import json from pathlib import Path
```

```
logging.basicConfig( level=logging.INFO, format='%(asctime)s [%
(levelname)s] %(message)s',
handlers=[ logging.FileHandler("data_processor.log", encoding='utf-8'),
logging.StreamHandler(sys.stdout) ] )
```

```
class DataProcessingError(Exception): pass
```

```

def load_json_file(filepath): path = Path(filepath) try: if not path.exists():
raise FileNotFoundError(f"      : {filepath}") with open(path, 'r',
encoding='utf-8') as f: data = json.load(f) assert isinstance(data, list),
"JSON      " logging.info(f"      {len(data)}      ") return data except
(json.JSONDecodeError, FileNotFoundError, AssertionError) as e:
logging.error(f"      : {e}") raise DataProcessingError(f"      : {str(e)}")
from e finally: logging.debug("      ")

```

```

def process_records(records): processed = [] for i, record in
enumerate(records): try: assert 'name' in record and 'score' in record, f"
{i}      " assert isinstance(record['score'], (int, float)), "      " if
record['score'] < 0: raise ValueError(f" {i}      ")
processed.append({ 'name': record['name'], 'score': record['score'],
'level': '  ' if record['score'] >= 90 else '  ' }) except Exception as e:
logging.warning(f" {i}      : {e}") continue return processed

```

```

if __name__ == "__main__": try: data = load_json_file("students.json")
result = process_records(data) logging.info(f"      {len(result)}      ")
print(json.dumps(result, ensure_ascii=False, indent=2)) except
DataProcessingError as e: logging.critical("      : " + str(e)) sys.exit(1)
except Exception as e: logging.critical("      ", exc_info=True)

```

students.json

9.9

```
1.         except: except Exception
2.     except     except: pass         logging
3.         with
4.             assert
5.         logging     print
6.                             debug
```

- finally Python 3 `__context__`
- bug
-
- `raise Exception("msg")`

9.10

“ ”

- Python Traceback
- try-except-else-finally

- raise
- print assert pdb IDE
- logging
-

-
- 1.
 2. logging
 3. logging
 4. pdb VS Code assert

4500

Python 3.8+

00P

4500

Linux

10

OOP

Python

9

Excel

“ ” —

Object-Oriented Programming

OOP

OOP

“ ”

install_engine() put_wheels() paint_color()..... 10

OOP

“ ”

Class

Django Flask Pygame pandas

OOP

“ ”

10.1

self

10.1.1

- **Class** “ ”

- **Object / Instance** “ ”

Dog

```
python class Dog: """          OOP  """ #          species = "Canis
lupus familiaris" def __init__(self, name, age): self.name = name #
self.age = age def bark(self): print(f"          {self.name}    {self.age}    ")
def eat(self, food): print(f"{self.name}    {food}    ")
```

```
dog1 = Dog(" ", 3) dog2 = Dog(" ", 5)
```

```
dog1.bark() dog2.eat(" ") print(f" {dog1.species}")
```

3

Canis lupus familiaris


```

    self

    def bark(self):
        """
        Python Dog.bark(dog1) dog1 self

        • self.name " "

        • self.age " "

        """
        return 30 " " self

1. class MyBeautifulDog
2.     self
3. __init__
4.     dog1.weight = 15 __init__

10.2 __init__

```

```

10.2.1 __init__

__init__ Dog("", 3) " "

```

```

python class Dog:
    def __init__(self, name, age=1, color=" "):
        self.name = name
        self.age = age
        self.color = color
        self.hunger = 100 #
    def bark(self):
        print(f"{self.name} {self.color} ")
    def eat(self, food="

```

```
) : self.hunger = max(0, self.hunger - 30) print(f"{self.name} {food} {self.hunger}")
```

```
dog3 = Dog(" ", 2, " ") dog3.bark() dog3.eat(" ") print(dog3.hunger)
```

```
__init__
```

10.2.2

" "

Python

- `_single_underscore` " "

- `__double_underscore` Name Mangling `_ClassName_attr`

```
python class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f" {amount} {self.__balance}")
    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f" {amount} {self.__balance}")
        else:
            print(" ")
    # def get_balance(self):
    return self.__balance
```

```
account = BankAccount(" ", 1000)
account.deposit(500)
account.withdraw(300)
print(" ", account.get_balance())
```

`print(account.__balance) #`

`AttributeError`

- Python “ ” _ “ ”
- `@property`

10.3

10.3.1 Inheritance

```
python class Animal: def __init__(self, name): self.name = name def
eat(self): print(f'{self.name}  ")
```

```
def sleep(self): print(f'{self.name}  ")
```

```
class Dog(Animal): # Animal def __init__(self, name, breed):
super().__init__(name) # self.breed = breed def bark(self):
print(f'{self.name} {self.breed}  ")
```

```
class Cat(Animal): def meow(self): print(f"{self.name} ~")
```

```
dog = Dog(" ", " ") cat = Cat(" ")
```

```
dog.eat() # dog.bark() cat.meow()
```

```
super()
```

10.3.2 Polymorphism

“ Python Duck Typing

```
python class Car: def run(self): print(" ")
```

```
class Airplane: def run(self): print(" ")
```

```
class Person: def travel(self, vehicle): print(" ...") vehicle.run() print("\n")
```

```
person = Person() person.travel(Car()) person.travel(Airplane())
```

run()

10.3.3 Magic Methods / Dunder Methods

Python

```
python class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages
    def __str__(self):
        """ print() """
        return f"{self.title} - {self.author} {self.pages} "
    def __repr__(self):
        return f"Book('{self.title}', '{self.author}', {self.pages})"
    def __len__(self):
        return self.pages
    def __add__(self, other):
        """ if isinstance(other, Book):
            return self.pages + other.pages
        return NotImplemented
```

```
book1 = Book("Python", " ", 320)
book2 = Book("Linux", " ", 680)
```

```
print(book1) # __str__
print(len(book1)) # __len__
print(book1 + book2)
```

ORM

10.4

```
python class Person: def __init__(self, name, age): self.name = name
self.age = age def __str__(self): return f"{self.name} {self.age}"
```

```
class Student(Person): def __init__(self, name, age, student_id):
super().__init__(name, age) self.__student_id = student_id #
self.courses = [] def add_course(self, course):
self.courses.append(course) print(f"{self.name} {course}") def
get_info(self): return f" {self.__student_id} {self.courses}" def
__len__(self): return len(self.courses)
```

```
class Teacher(Person): def __init__(self, name, age, subject):
super().__init__(name, age) self.subject = subject def teach(self):
print(f"{self.name} {self.subject}")
```

```
students = [] s1 = Student("  ", 18, "2023001") s1.add_course("Python")  
s1.add_course("  ") print(s1) print(s1.get_info()) print("  ", len(s1))
```

```
teacher = Teacher("  ", 35, "OOP") teacher.teach()
```


10.5

-
- `self` `__init__`
-
- `" Python "`
- OOP

1. `Car` `drive(km)`
2. `ElectricCar` `Car`
3. `Car` `__str__` `__add__`
4. `Library` `__getitem__` `__len__`
- 5.

OOP

5200

+ +

10

Python

11

11

Python

11 —

“ ”

.....

Linux

/bin /usr/bin /etc

Python

Mo

Python

“Batteries Included”

-
- `__name__`
- `__init__.py`
- `pip`

- os sys time datetime random

“ ” “ ”

11.1

11.1.1

.py

.py

- 1.
- 2.
- 3.

11.1.2

Python

math

python

1

```
import math
```

```
print(math.pi) # 3.141592653589793 print(math.sqrt(16)) # 4.0
```



```
python
```

2

```
from math import sqrt, pi
```

```
print(pi) print(sqrt(25))
```



```
python
```

3 as

```
import math as m
```

```
print(m.pow(2, 3)) # 8.0
```



python

4

```
from math import *
```

```
print(sin(0)) # 0.0 print(cos(0)) # 1.0
```

- `from module import *`
-
- `import os, sys, time`

11.1.3

```
import xxx Python
```

- 1.
 2. PYTHONPATH
 - 3.
 4. `site-packages`
- `sys`

```
python import sys print(sys.path)
```

ModuleNotFoundError

11.2 `__name__`

11.2.1

`utils.py`

python

utils.py

```
def add(a, b): """        """ return a + b
```

```
def multiply(a, b): """        """ return a * b
```

```
def print_info(name, age): print(f"    {name}    {age}")
```

```
if __name__ == "__main__": print("    utils.py    ") print("5 + 3 =",  
add(5, 3)) print("4 * 7 =", multiply(4, 7)) print_info("  ", 18)
```

```
main.py
```

```
python
```

main.py

```
import utils
```

```
print(" ") print("10 + 20 =", utils.add(10, 20)) utils.print_info(" ",  
25)
```

```
python main.py      utils.py
```

11.2.2 `__name__`

Python `__name__`

- `__name__` `"__main__"`
- `__name__` `"utils"`

```
if __name__ == "__main__":
```

11.3 Package

`__init__.py` Python 3.3+ `__init__.py`

11.3.1

```
myproject/ ├── main.py ├── utils/ | ├── __init__.py | ├── math_utils.py |  
           ├── file_utils.py | └── __pycache__/ └── tools/ ├── __init__.py └──  
           network_utils.py
```

```
__init__.py      __all__      from package import *
```

python

utils/__init__.py

```
__all__ = ['math_utils', 'file_utils']
```

```
print("utils      ") #
```



```
python
```

```
from utils.math_utils import add
```

```
from .math_utils import add #      from ..tools.network_utils import * #
```

11.4 pip

Python pip

11.4.1 pip

bash

```
pip install requests pip install -i https://pypi.tuna.tsinghua.edu.cn/simple  
requests #
```

```
pip install pandas==2.0.0
```

pip uninstall requests

```
pip list pip freeze > requirements.txt #
```

```
pip install -r requirements.txt
```

```
venv
```

```
bash python -m venv venv
```

Windows: venv\Scripts\activate

Linux/macOS: source venv/bin/ activate

11.5

11.5.1 os — “Linux”

os Linux ls mkdir rm

python import os

1.

```
print(os.getcwd()) #
```

```
print(os.path.join('folder', 'file.txt')) #
```


2.

```
if not os.path.exists('test_dir'): os.mkdir('test_dir')
```

3.

```
def list_all_files(directory): for root, dirs, files in os.walk(directory): print(f"
    : {root}") for file in files: print(f"    : {os.path.join(root, file)}") for d in
dirs: print(f"    : {d}")
```

4.

```
def batch_rename(directory, prefix="new_"): for filename in
os.listdir(directory): if filename.endswith('.txt'): old_path =
os.path.join(directory, filename) new_path = os.path.join(directory, prefix
+ filename) os.rename(old_path, new_path) print(f"    : {filename} ->
{prefix + filename}")
```

```
11.5.2 sys — Python
```

```
python import sys
```

```
print("Python :", sys.version) print("    :", sys.platform)
```

```
if len(sys.argv) > 1: print("      :", sys.argv[1:]) else: print("      ")
```

sys.exit(0) # 0

```
11.5.3 time    datetime —
```

```
python import time import datetime
```

time

```
print("      :", time.time()) time.sleep(1) # 1 print("      :",  
time.strftime("%Y-%m-%d %H:%M:%S"))
```

datetime

```
now = datetime.datetime.now() print("    :", now) print("    :",  
now.weekday() + 1) # 0=
```

```
birthday = datetime.date(2000, 1, 1) today = datetime.date.today() delta
= today - birthday print(f"      {delta.days}  ")
```

```
def countdown(seconds): for i in range(seconds, 0, -1): print(f"    : {i}  ",
end="\r") time.sleep(1) print("    ")
```

11.5.4 random —

```
python import random
```

```
print("    1-100:", random.randint(1, 100)) print("    :",
random.random()) print("    :", random.choice([' ', ' ', ' ', ' ']))
```

```
cards = ['A', '2', '3', '4', '5', 'K'] random.shuffle(cards) print("    :", cards)
```

```
def lottery_draw(participants, num_winners=3): winners =
random.sample(participants, num_winners) print("      ") for i, winner
in enumerate(winners, 1): print(f" {i}    {winner}") return winners
```

```
users = ["  ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  ", "  "] lottery_draw(users, 2)
```

11.6

```
python import os import shutil import datetime import random
```

```
def organize_files(source_dir): """          """ categories = { ' ':
['.jpg', '.png', '.gif', '.jpeg'], ' ': ['.pdf', '.docx', '.xlsx', '.txt'], ' ': ['.py',
'.js', '.html', '.css'], ' ': [] } for filename in os.listdir(source_dir): if
filename.startswith('.'): #          continue file_path =
os.path.join(source_dir, filename) if os.path.isfile(file_path): #          ext
= os.path.splitext(filename)[1].lower() #          category = ' ' for cat, exts
in categories.items(): if ext in exts: category = cat break #
today = datetime.datetime.now().strftime('%Y-%m-%d') target_dir =
os.path.join(source_dir, category, today) os.makedirs(target_dir,
exist_ok=True) #          target_path = os.path.join(target_dir,
```

```
filename) shutil.move(file_path, target_path) print(f"    : {filename} ->  
{category}/{today}/") time.sleep(random.uniform(0.1, 0.5)) #
```

```
if __name__ == "__main__": organize_files("./    ") print("    ")
```

```
os shutil datetime random
```

11.7

```
1.         +
2.
3.     if __name__ == "__main__":
4.
5.         requirements.txt +
```

- A B B A
-
- ModuleNotFoundError
- __init__.py

Python

os sys time datetime random

“ ”

```
1.     logger.py         __name__
2.     os.walk           .py
```

```
3. random datetime " "
4. mypackage __init__.py
```

6500

—

11

12

—

11 print("Hello, World!")

" "

```
1. file_renamer.py
2. student_manager.py
```

PEP 8

--help

12.1

Command Line Interface CLI

Python CLI **argparse**

12.1.1 argparse



```
python import argparse
```

```
def main(): parser = argparse.ArgumentParser( description="          ",
epilog="          " ) parser.add_argument('-n', '--name', default=' ',
help='          ') parser.add_argument('-c', '--count', type=int, default=1,
help='          ') args = parser.parse_args() for _ in range(args.count): print(f"
{args.name}          Python          ")
```

```
if __name__ == "__main__": main()
```



```
bash python greet.py --help python greet.py -n          -c 3
```


ArgumentParser

add_argument

parse_args()

Namespa

“

”

description epilog

--help

argparse

- pathlib os.path

- json

- logging print()

- re

- OOP

12.2

DSC_0001.jpg

12.2.1

-

- 001 002...

-

-
-
- `--dry-run`
-
-
-
-
- CLI
-
- `pathlib.Path` `os`
-
- PEP 8 4

file_renamer/ ├── file_renamer.py # ├── README.md └── examples/ #

12.2.2

180

python #!/usr/bin/env python3

-- coding: utf-8 --

""" Python 1.0 """

import argparse import logging import re import sys from pathlib import
Path from typing import List, Optional

```
logging.basicConfig( level=logging.INFO, format='%(asctime)s - %
(levelname)s - %(message)s', datefmt='%H:%M:%S' ) logger =
logging.getLogger(__name__)
```

```
class FileRenamer: """          OOP          """ def __init__(self, dry_run:
bool = True, backup: bool = True): self.dry_run = dry_run self.backup =
backup self.renamed_count = 0 self.skipped_count = 0 def get_files(self,
directory: Path, recursive: bool = False, extensions: Optional[List[str]] =
None) -> List[Path]: """          """ if recursive: files =
list(directory.rglob("*")) else: files = list(directory.glob("*")) #
files = [f for f in files if f.is_file()] if extensions: ext_set = {ext.lower() if
ext.startswith('.') else '.' + ext.lower() for ext in extensions} files = [f for f
in files if f.suffix.lower() in ext_set] return sorted(files) def
generate_new_name(self, old_path: Path, prefix: str = "", suffix: str = "",
numbering: bool = False, number_start: int = 1, regex_pattern: str = "",
regex_replace: str = "", index: int = 0) -> str: """          """ name =
old_path.stem ext = old_path.suffix #          if regex_pattern: try:
name = re.sub(regex_pattern, regex_replace, name) except re.error as e:
logger.error(f"          : {e}") return None #          if prefix: name = prefix
+ name if suffix: name = name + suffix #          if numbering: name =
f"{name}_{number_start + index:03d}" new_name = name + ext return
new_name def rename_files(self, directory: Path, prefix: str = "", suffix: str
= "", numbering: bool = False, number_start: int = 1, regex_pattern: str =
"", regex_replace: str = "", recursive: bool = False, extensions:
Optional[List[str]] = None, confirm: bool = True) -> None: """          """
files = self.get_files(directory, recursive, extensions) if not files:
logger.info("          ") return logger.info(f"    {len(files)}    ") #
if self.dry_run: print("\n===          ===") for i, old_path in
enumerate(files): new_name = self.generate_new_name( old_path, prefix,
```

```

suffix, numbering, number_start, regex_pattern, regex_replace, i ) if
new_name: print(f"{old_path.name} --> {new_name}") print("\n      --
no-dry-run          ") return #          if confirm: answer = input("\n
                        (y/N): ") if answer.lower() != 'y': logger.info("          ") return
for i, old_path in enumerate(files): new_name =
self.generate_new_name( old_path, prefix, suffix, numbering,
number_start, regex_pattern, regex_replace, i ) if not new_name:
self.skipped_count += 1 continue new_path = old_path.parent /
new_name #          if new_path.exists() and new_path != old_path:
logger.warning(f"          : {new_name}") self.skipped_count += 1
continue try: old_path.rename(new_path) logger.info(f"          :
{old_path.name} -> {new_name}") self.renamed_count += 1 #
    if self.backup: backup_file = old_path.parent / ".rename_backup.txt"
with open(backup_file, "a", encoding="utf-8") as f:
f.write(f"{new_path.name} <-- {old_path.name}\n") except Exception as
e: logger.error(f"          {old_path.name}: {e}") self.skipped_count += 1
logger.info(f"          {self.renamed_count}          {self.skipped_count}
")

```

```

def main(): parser = argparse.ArgumentParser( description="
", formatter_class=argparse.RawDescriptionHelpFormatter, epilog="""
    %(prog)s . --prefix " A_" --numbering --dry-run %(prog)s photos/ --
regex "\d+" "IMG" --no-dry-run %(prog)s . --suffix "_final" --ext jpg png --
recursive """) parser.add_argument("directory", type=Path, help="
", nargs="?", default=Path(".")) parser.add_argument("-p", "--
prefix", type=str, default="", help="          ") parser.add_argument("-s",
"--suffix", type=str, default="", help="          ")
parser.add_argument("-n", "--numbering", action="store_true", help="
") parser.add_argument("--start", type=int, default=1, help="
1") parser.add_argument("-r", "--regex", nargs=2, metavar=("PATTERN",
"REPLACE"), help="          ") parser.add_argument("--recursive", "-
R", action="store_true", help="          ") parser.add_argument("--ext",
nargs="+", help="          jpg png ") parser.add_argument("--no-dry-

```

```

run", dest="dry_run", action="store_false", default=True, help="
    ") parser.add_argument("--no-backup", dest="backup",
action="store_false", default=True, help="    ") args =
parser.parse_args() if not args.directory.exists(): logger.error("    ")
sys.exit(1) renamer = FileRenamer(dry_run=args.dry_run,
backup=args.backup) regex_pattern = args.regex[0] if args.regex else ""
regex_replace = args.regex[1] if args.regex else ""
renamer.rename_files( directory=args.directory, prefix=args.prefix,
suffix=args.suffix, numbering=args.numbering, number_start=args.start,
regex_pattern=regex_pattern, regex_replace=regex_replace,
recursive=args.recursive, extensions=args.ext, confirm=not
args.dry_run )

if __name__ == "__main__": main()

```

`file_renamer.py`

`chmod +x file_renamer.py`

Linux/macOS

12.2.3

```
1. FileRenamer GUI " "___
2. get_files pathlib rglob os.walk Pythonic
3. generate_new_name :03d 001 002
4. rename_files try-except
5. main argparse RawDescriptionHelpFormatter --regex PATTERN
```

- `file_renamer.py 2023_/ --prefix "_" --numbering --ext jpg`
- `file_renamer.py ~/Downloads --regex " \\(\\d+\\)" "" --no-dry-run`
- `--dry-run`
-
-

12.3

" "

OOP JSON

CLI

12.3.1

- / /

-

-

- students.json

- argparse

Student + GradeManager

12.3.2 student_manager.py

```
python import json import logging from pathlib import Path from typing
import Dict, List, Optional
```

```
logging.basicConfig(level=logging.INFO, format='%(message)s') logger =
logging.getLogger(__name__)
```

```
class Student: def __init__(self, name: str, scores: Dict[str, int]): self.name
= name self.scores = scores # {" ": 85, " ": 92, ...} def
get_average(self) -> float: if not self.scores: return 0.0 return
sum(self.scores.values()) / len(self.scores) def to_dict(self) -> dict: return
{"name": self.name, "scores": self.scores} @classmethod def
from_dict(cls, data: dict): return cls(data["name"], data["scores"])
```

```

class GradeManager:
    def __init__(self, data_file: str = "students.json"):
        self.data_file = Path(data_file)
        self.students: List[Student] = []

    def load_data(self):
        if self.data_file.exists():
            try:
                with open(self.data_file, "r", encoding="utf-8") as f:
                    data = json.load(f)
                self.students = [Student.from_dict(item) for item in data]
                logger.info(f"Loaded {len(self.students)} students")
            except Exception as e:
                logger.error(f"Error loading data: {e}")

    def save_data(self):
        try:
            with open(self.data_file, "w", encoding="utf-8") as f:
                json.dump([s.to_dict() for s in self.students], f, ensure_ascii=False, indent=2)
            logger.info(f"Saved {len(self.students)} students")
        except Exception as e:
            logger.error(f"Error saving data: {e}")

    def add_student(self, name: str, scores: Dict[str, int]):
        # Check if student already exists
        if any(s.name == name for s in self.students):
            logger.warning(f"Student {name} already exists")
            self.update_student(name, scores)
        else:
            self.students.append(Student(name, scores))
        self.save_data()
        logger.info(f"Added student {name}")

    def update_student(self, name: str, scores: Dict[str, int]):
        for student in self.students:
            if student.name == name:
                student.scores.update(scores)
                self.save_data()
                logger.info(f"Updated student {name}")
                return
        logger.error(f"Student {name} not found")

    def delete_student(self, name: str):
        original_len = len(self.students)
        self.students = [s for s in self.students if s.name != name]
        if len(self.students) < original_len:
            self.save_data()
            logger.info(f"Deleted student {name}")
        else:
            logger.warning(f"Student {name} not found")

    def query_student(self, name: str) -> Optional[Student]:
        for student in self.students:
            if student.name == name:
                return student
        return None

    def show_statistics(self):
        if not self.students:
            print("No students found")
            return
        print("\n=====")
        print(f"Total students: {len(self.students)}")
        all_averages = [s.get_average() for s in self.students]
        print(f"Average scores: {sum(all_averages)/len(all_averages):.2f}")
        print(f"Max score: {max(all_averages):.2f}")
        print(f"Min score: {min(all_averages):.2f}")
        # Pass count
        pass_count = sum(1 for avg in all_averages if avg >= 60)
        print(f"Pass count: {pass_count/len(self.students)*100:.1f}%")
        print("=====")

    def list_all(self):
        if not self.students:
            print("No students found")
            return
        print("\n{<10} {<8} {<8} {<8}".format("Name", "Math", "Science", "Average"))
        print("-" * 40)
        for s in self.students:
            print(s)

```

```

math = s.scores.get("  ", 0) english = s.scores.get("  ", 0)
print(f"{s.name:<10} {math:<8} {english:<8} {s.get_average():<8.1f}")

```

```

def main_menu(): manager = GradeManager() while True: print("\n" +
"*40) print(" ") print("=*40) print("1.  / ") print("2. ")
print("3. ") print("4. ") print("5. ") print("6. ")
print("=*40) choice = input(" (1-6): ").strip() if choice == "1": name
= input(" : ") try: math = int(input(" : ")) english = int(input(" : "))
manager.add_student(name, {" ": math, " ": english}) except
ValueError: logger.error(" ") elif choice == "2": name = input("
: ") student = manager.query_student(name) if student: print(f"\n :
{student.name}") for subject, score in student.scores.items():
print(f" {subject}: {score}") print(f" : {student.get_average():.2f}")
else: print(" ") elif choice == "3": name = input(" : ")
manager.delete_student(name) elif choice == "4": manager.list_all() elif
choice == "5": manager.show_statistics() elif choice == "6": print("
") break else: print(" ")

```

```

if __name__ == "__main__": main_menu()

```

python student_manager.py

students.json

- Student
- GradeManager “ ”
- try...except ValueError
- JSON

Excel pandas

argparse

add/query/sta

12.4

12.4.1 PEP 8

- 4
- snake_case PascalCase
- 79
- docstring
-

12.4.2

```
my_tool/ ├── my_tool/ | ├── __init__.py | ├── cli.py #       | ├── core.py
#       | ├── utils.py ├── tests/ ├── README.md ├── pyproject.toml ├──
LICENSE`
```

12.4.3

- **print** logging DEBUG/INFO/WARNING/ERROR
- except:
- from typing import List
- **dry-run**
- unittest
- PyInstaller exe Python
- Windows Linux pathlib
- encoding="utf-8"
- rename
- root

12.5

"

GitHub

file_renamer.py

GitHub

- Python argparse
- PEP 8
- pathlib